

Spring boot - 2

■ Spring in Action 5판 기반 스프링 부트 공부 필기

참고

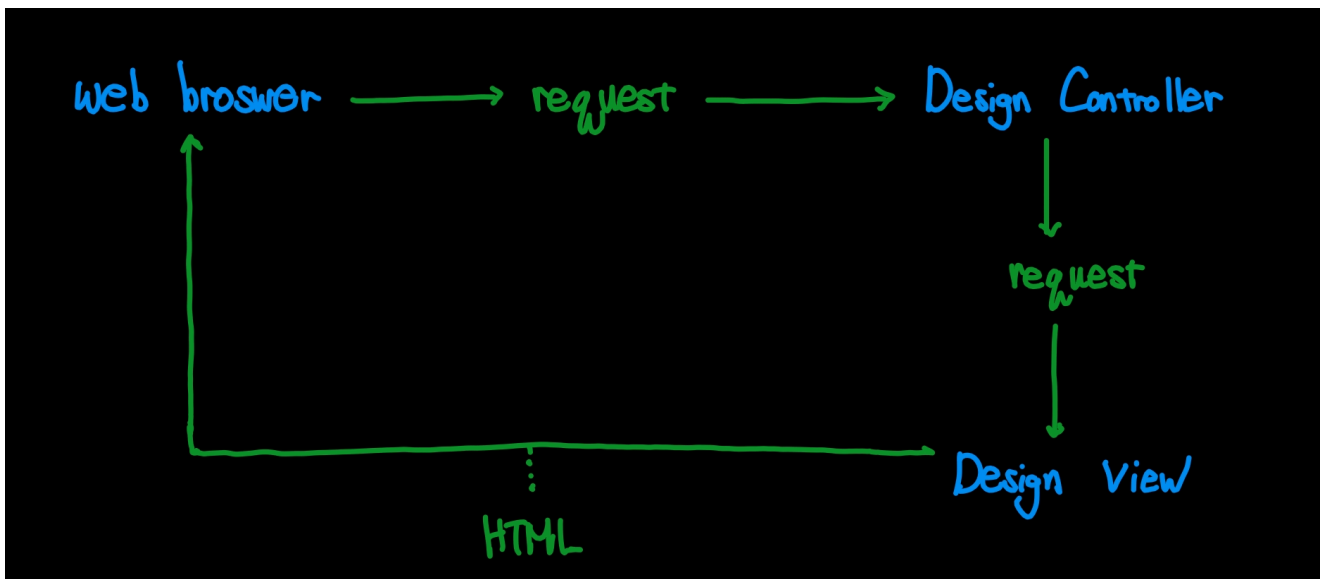
- 책: *Spring in Action* 제5판 / 크레이그 윌즈 지음, 심재철 옮김
- 코드 및 예시는 책 내용을 기반으로 함
- 나의 학습 환경: Java 17, IntelliJ
- PDF 다운로드: [<https://plusb3b.kr/pdf-download-page/index.html>]
- 책의 공식 GitHub: <https://github.com/Jpub/SpringInAction5>

🎯 1. 정보 보여주기

- HTML Page: 하드코딩 ✗
- Controller: 데이터를 가져와 View로 전달

구성요소

1. 도메인 클래스
2. Spring MVC Controller
3. View Template



📦 도메인 설정하기

애플리케이션 도메인

데이터 타입, 이름, 식별 가능한 ID를 다루는 개념

🍎 Ingredient 클래스

(공식 예시와 다를 수 있음)

```
package tacos;
```

```
import lombok.Data;
```

```
import lombok.RequiredArgsConstructor;
```

```
import java.lang.reflect.Type;
```

@Data : final 속성 추가와 생성자 + getter, setter 생성

① RequiredArgsConstructor

```
public class Ingredient {
```

```
    private final String id;
```

```
    private final String name;
```

```
    private Type type; → 정해진 상수 값 저장, 변경 x, memory 사용량 ↓
```

```
    public static enum Type {
```

```
        WRAP, PROTIEN, VEGGIES, CHEESE, SAUCE
```

```
    }
```

```
    public Ingredient () { }
```

```
    public Ingredient (String id, String name, Type type) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
        this.type = type;
```

```
    }
```

getter method / setter method

🌮 Taco 클래스

```
1    package tacos;
2    import java.util.List;
3
4    import javax.validation.constraints.NotNull;
5    import javax.validation.constraints.Size;
6    import lombok.Data;
7
8    @Data
9    ✓ public class Taco {
10
11        @NotNull
12        @Size(min=5, message="Name must be at least 5 characters long")
13        private String name;
14
15        @Size(min=1, message="You must choose at least 1 ingredient")
16        private List<String> ingredients;
17    }
```

Controller 생성

| Spring MVC의 핵심: 요청 처리 + View 반환

기능

- /design 경로 GET 요청 처리
- 모델 데이터 생성
- View 템플릿 렌더링 요청
- 결과 HTML을 브라우저에 전송

```

19  @Slf4j
20  @Controller
21  @RequestMapping("/design")
22  ✓ public class DesignTacoController {
23      @GetMapping
24      ✓ public String showDesignForm(Model model) {
25          List<Ingredient> ingredients = Arrays.asList(
26              new Ingredient("FLT0", "Flour Tortilla", Type.WRAP),
27              new Ingredient("COTO", "Corn Tortilla", Type.WRAP),
28              new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
29              new Ingredient("CARN", "Carnitas", Type.PROTEIN),
30              new Ingredient("TMT0", "Diced Tomatoes", Type.VEGGIES),
31              new Ingredient("LETC", "Lettuce", Type.VEGGIES),
32              new Ingredient("CHED", "Cheddar", Type.CHEESE),
33              new Ingredient("JACK", "Monterrey Jack", Type.CHEESE),
34              new Ingredient("SLSA", "Salsa", Type.SAUCE),
35              new Ingredient("SRCR", "Sour Cream", Type.SAUCE)
36          );
37
38          Type[] types = Ingredient.Type.values();
39          for (Type type : types) {
40              model.addAttribute(type.toString().toLowerCase(),
41                  filterByType(ingredients, type));
42          }
43
44          model.addAttribute("taco", new Taco());
45
46          return "design";
47      }
48
49      ✓ private List<Ingredient> filterByType(
50          List<Ingredient> ingredients, Type type) {
51          return ingredients
52              .stream()
53              .filter(x -> x.getType().equals(type))
54              .collect(Collectors.toList());
55      }
56
57      @PostMapping
58      ✓ public String processDesign(@Valid Taco design, Errors errors) {
59          if (errors.hasErrors()) {
60              return "design";
61          }
62
63          // 이 지점에서 타코 디자인(선택된 식자재 내역)을 저장한다...
64          // 이 작업은 3장에서 할 것이다.
65          log.info("Processing design: " + design);
66          return "redirect:/orders/current";
67      }
68  }

```

어노테이션 설명

- @Slf4j : Lombok 제공 로깅 지원
- @Controller : 스프링이 Bean으로 인식
- @RequestMapping : 클래스 레벨 경로 매핑
- @GetMapping : GET 요청 메서드 지정

showDesignForm(), filterByType()

→ View (html 페이지)에서 사용할 데이터 담은 객체.
- showDesignForm (Model 객체) Get 요청 들어오면 처리.

```
List<Ingredient> ingredients = Arrays.asList(  
    new Ingredient("FLTO", "Flour Tortilla", Type.WRAP)  
    ...  
);
```

① Ingredient 객체
저장하는 List 객체 생성

Type[] types = Ingredient.Type.values(); : enum Type 모든 값 가져온다.

for (Type type : types)

model.addAttribute(type.toString().toLowerCase(), filterByType(ingredient, type));
문자열로 변환 후, 소문자로 변환

model.addAttribute("taco", new Taco()); : model.addAttribute("키", "값") 형식으로
데이터 쿼리하면 사용 가능

return "design";

Model : 컨트롤러와 데이터 보여주는 View 사이에서 데이터 운반하는 객체.

```
private List<Ingredient> filterByType (List<Ingredient> ingredients, Type type) {
```

return ingredients.stream() : ingredients 리스트를 stream으로 변환.

.filter(x -> x.getType().equals(type)) : type과 일치하는 자료만 선택

.collect(Collectors.toList()); : 필터링된 개체를 새로운 리스트로 변환.

}

View 디자인

- JSP, Thymeleaf, FreeMarker 등 사용 가능
- **Thymeleaf**: HTML 요소에 속성 추가

design.html 은 1장 참고

2. 폼 제출 처리하기

```

<form method="POST" th:object="${taco}">
    <span class="validationError"
        th:if="${#fields.hasErrors('ingredients')}}"
        th:errors="*{ingredients}">Ingredient Error</span>

    <div class="grid">
        <div class="ingredient-group" id="wraps">
            <h3>Designate your wrap:</h3>

            <div th:each="ingredient : ${wrap}">
                <input name="ingredients" type="checkbox"
                    th:value="${ingredient.id}" />
                <span th:text="${ingredient.name}">INGREDIENT</span><br />
            </div>
        </div>

        <div class="ingredient-group" id="proteins">
            <h3>Pick your protein:</h3>
            <div th:each="ingredient : ${protein}">

```

POST 요청 메서드 작성

```

@PostMapping
public String processDesign(@Valid Taco design, Errors errors) {
    if (errors.hasErrors()) {
        return "design";
    }

    // 이 지점에서 타코 디자인(선택된 식자재 내역)을 저장한다...
    // 이 작업은 3장에서 할 것이다.
    log.info("Processing design: " + design);
    return "redirect:/orders/current";
}

```

@PostMapping → /design = POST 요청 처리
 public String processDesign(Taco design) {
 [타코 디자인 내역 저장]
 log.info("Processing design : " + design);
 return "redirect : /orders/current";
 }

processDesign() 실행 끝난 후, 사용자 브라우저가
 /orders/current 상단 경로로 재접속

💡 form에 action 생략 시

→ /design 경로로 POST 요청됨

로그 설정

```
private static final org.slf4j.Logger log =  
    org.slf4j.LoggerFactory.getLogger(DesignTacoController.class);
```

/order/current Controller 필요

```
@Slf4j  
@Controller  
@RequestMapping("/orders")  
public class OrderController {  
  
    @GetMapping("/current")  
    public String orderForm(Model model) {  
        model.addAttribute("order", new Order());  
        return "orderForm";  
    }  
}
```

```
@Controller  
@RequestMapping("/orders") ← /orders로 시작되는 경로 요청 처리.  
public class OrderController {  
  
    @GetMapping("/current") ← /orders/current 경로 GET 요청 처리.  
    public String orderForm (Model model) {  
        model.addAttribute("order", new Order());  
        return "orderForm"; ← view 이름 반환  
    }  
}
```

OrderForm 및 Order 클래스

웹에서는 오류 안나지만

```
<label for = "deliveryName" > Name: </label>  
<input type = "text" id = "deliveryName" th:field = "${deliveryName}" >  
    적어줘야 오류 안난다.
```

여기서는 (`<form th:action = "@{/orders}" ... >`) → HTTP POST 요청
액션지정

↓
@PostMapping
public String processOrder(Order order)
{
 log.info(———);
 return "redirect : /design";
}
/design으로 리다이렉트.

```
@Slf4j  
@Controller  
@RequestMapping("/orders")  
public class OrderController {  
  
    @GetMapping("/current")  
    public String orderForm(Model model) {  
        model.addAttribute("order", new Order());  
        return "orderForm";  
    }  
  
    @PostMapping  
    public String processOrder(@Valid Order order, Errors errors) {  
        if (errors.hasErrors()) {  
            return "orderForm";  
        }  
  
        log.info("Order submitted: " + order);  
        return "redirect:/";  
    }  
}
```

✓ 3. 폼 유효성 검사하기

누락 필드, 길이 제한 등 입력값 검증

Spring MVC 검증 흐름

1. 도메인 클래스에 어노테이션 선언
2. Controller 메서드에 @Valid 사용
3. View에 에러 표시

Taco 클래스 예시

import 문 (JDK 버전에 따라)

- JDK 8~11: javax.validation.constraints.*
- JDK 17+: jakarta.validation.constraints.*

```
<dependency>
  <groupId> org.springframework.boot </groupId>
  <artifactId> spring-boot-starter-validation </artifactId>
</dependency>
```

@Data

```
public class Taco {

    @NotNull
    @Size(min=5, message="Name must be at least 5 characters long")
    private String name;

    @Size(min=1, message="You must choose at least 1 ingredient")
    private List<String> ingredients;
}
```

Order 클래스

- 주소 필드: @NotBlank
- 카드 필드: @CreditCardNumber, @Pattern, @Digits

```
import jakarta.validation.constraints.*;
import org.hibernate.validator.constraints.CreditCardNumber;
```

⋮

@NotBlank(message=" ~ ") =>

private String deliveryStreet;

⋮

@CreditCardNumber(message=" ~ ") : Luhn 알고리즘 검사에 합격한 신용카드 번호.

private String ccNumber;

(*) 사용가능한 자판단 : 금회망 연동.

⋮

@Pattern(regexp="^(0[1-9]|1[0-2])([\\/])([1-9][0-9])\$", message=" ~ ")

private String ccExpiration; : 장래 표현식 지정.

@Digits(integer=3, fraction=0, message=" ~ ") : Digits 지정하여 입력 값이 정각이

private String ccCW;

세 자리 숫자인지 검사.

```
@NotBlank(message="Name is required")
```

```
private String deliveryName;
```

```
@NotBlank(message="Street is required")
```

```
private String deliveryStreet;
```

```
@NotBlank(message="City is required")
```

```
private String deliveryCity;
```

```
@NotBlank(message="State is required")
```

```
private String deliveryState;
```

```
@NotBlank(message="Zip code is required")
```

```
private String deliveryZip;
```

```
@CreditCardNumber(message="Not a valid credit card number")
```

```
private String ccNumber;
```

```
@Pattern(regexp="^(0[1-9]|1[0-2])([\\/])([1-9][0-9])$",
```

```
message="Must be formatted MM/YY")
```

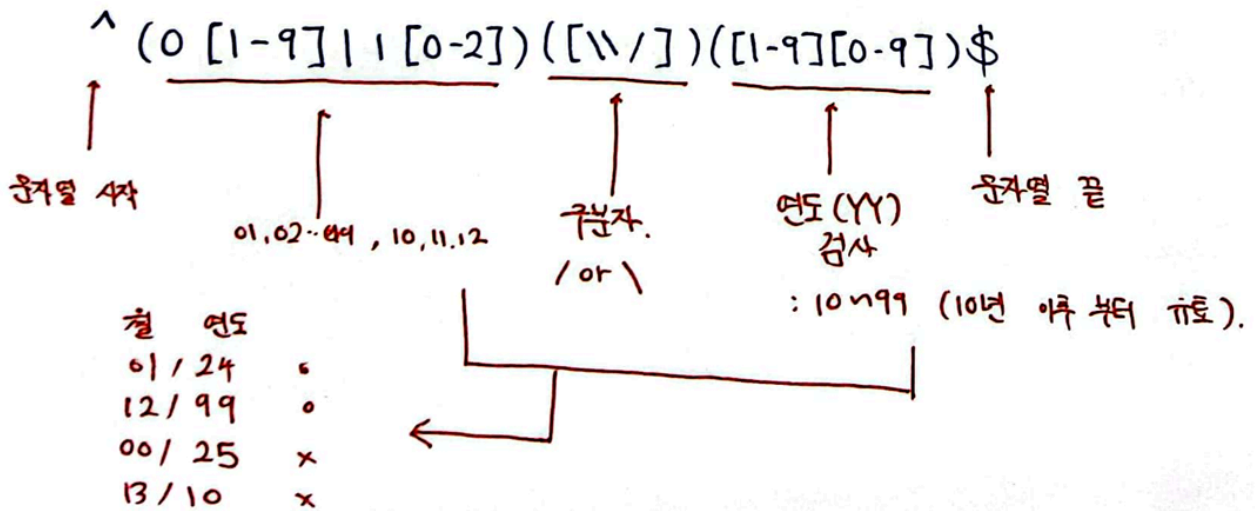
```
private String ccExpiration;
```

```
@Digits(integer=3, fraction=0, message="Invalid CW")
```

```
private String ccCW;
```

정규 표현식 예시

<정규 표현식>



@Valid 처리 예시

```
import jakarta.validation.Valid;
import org.springframework.validation.Errors;
```

- DesignTacoController

②. 작성.

@PostMapping

```
public String processDesign (@Valid Taco design, Errors errors) {
    if (errors.hasErrors()) return "design";
}
```

```
log.info(" ~ ");
return "redirect : /orders/current";
}
```

제출된 Taco 객체 검증 수행 하려고 Spring MVC에 알려줌.
제출된 폼 데이터와 Taco 객체가 바인딩 된 후, 해당 메소드 코드
실행 되기 전.

L, 어떠한 검사라도 에러 있다면
상세 내역이 Errors 객체에 저장되고
processDesign()으로 전달.
에러 있다면 "design" 뷰 이름 반환.
hasErrors() 호출하여, 에러 검사.

- OrderController

```
@PostMapping
public String processOrder(@Valid Order order, Errors errors) {
    if (errors.hasErrors()) {
        return "orderForm";
    }

    log.info("Order submitted: " + order);
    return "redirect:/";
}
```

오류 메시지 처리 (Thymeleaf)

```
<span th:if="${#fields.hasErrors('field')}" th:errors="*{field}"></span>
```

=> 무엇을 수정해야 하는지 사용자가 어떻게 알까?

L, 'th: errors' 설정 이용'

```
<label for="ccNumber">Credit Card #: </label>
```

```
<input type="text" id="ccNumber" th:field="*{ccNumber}">
```

```
<span class="Validation Error" th:if="${#fields.hasErrors('ccNumber')}"
      th:errors="*{ccNumber}">CC Num Error</span>
```

사용자 주시 끌기 위해
에러 명칭 지정하는데 사용.

hasErrors() 메소드로 ccNumber
필드에 에러 있는지 검사,

에러 O : 보여줌
에러 X : 안보임.

4. 뷰 컨트롤러로 작업하기

- 간단한 View 연결만 필요할 경우 사용
- 모델/입력 없이 View만 전달

WebConfig 설정 예

```
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
```

Springer 설정 클래스는 적당,

@Configuration → View Controller 역할 수행하는 구성 클래스. * WebConfigurer 구현 *

```
public class WebConfig implements WebConfigurer {
    : WebConfigurer 인터페이스 Spring MVC 구성하는 메소드 정의한다.
```

@Override

→ 하나 이상의 뷰 컨트롤러를 등록하기 위해 ViewControllerRegistry를 반환 받음

```
public void addViewControllers(ViewControllerRegistry registry) {
```

```
    registry.addViewController("/").setViewName("home");
```

→ 객체를 통해 URL과 View 매핑.

```
}
```

" / " 요청들어오면

home 이라는 View 보여줌도록한다.

❁ 5. View 템플릿 엔진 선택

템플릿	의존성
FreeMarker	spring-boot-starter-freemarker
Groovy	spring-boot-starter-groovy-templates
Mustache	spring-boot-starter-mustache
Thymeleaf	spring-boot-starter-thymeleaf
JSP	❌ (WAR 파일만 지원)

☹ JSP 사용 주의사항

- 내장 톰캣/Jetty는 /WEB-INF 경로 탐색 불가
- 실행 가능한 JAR에서는 JSP 사용 곤란 → WAR 패키징 필요

⚙ 템플릿 캐싱

- 기본: 최초 1회만 파싱 → 캐시 저장
- 개발 시 캐시 ❌ 필요 → DevTools 사용 권장

2. 웹 애플리케이션 개발

1) 정보 보이기.

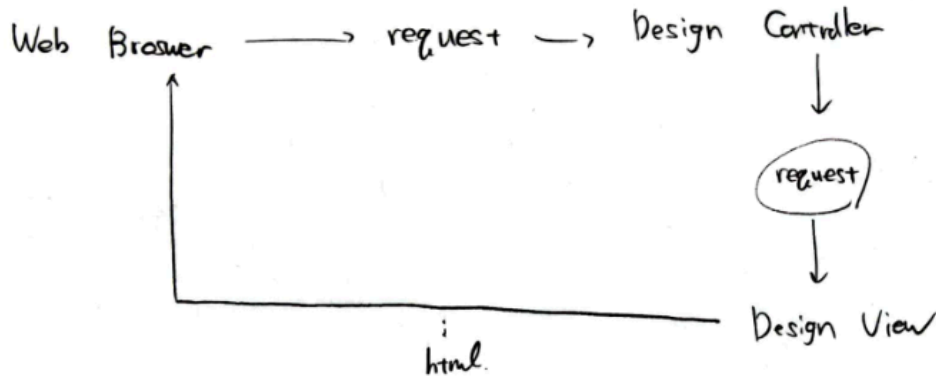
- html page : 하드 코딩 x.

- 데이터 가져오고 처리: Controller 역할 $\Rightarrow$ View 이 보여줌.

디자인 패턴 지지자 $\Rightarrow$ 소싱 정지 : Domain Class

View 이 전달 : Spring MVC Controller Class

View 템플릿



1-2) 도메인 설정

애플리케이션 도메인 $\Rightarrow$ 애플리케이션 이해야 필드인 개념 다중.

<MVC 흐름>

~~사용자~~

데이터 타입, 이름, 식별 가능한 ID.

```
import lombok.Data;
import lombok.RequiredArgsConstructor;
import java.lang.reflect.Type;
```

① Data final 속성 ~~최소 생성자~~ ~~getter~~ getter/setter method x
② RequiredArgsConstructor + getter equals(), hashCode(), toString() x) 도메인 클래스
public class Ingredient { setter 생성
라고 알려줌.
private final String id;

⋮
" Type type;

$\rightarrow$ 정해진 상수 값 저장, 변경 x, 메모리 사용량 ↓